GNU-AWK/AWK Cheatsheet

AWK BASICS

```
$ awk -F ":"  '{print $1, $6, $NF}' /etc/passwd
```

-F ":"	option sets the field separator to ":"
'{...}'	block inside single quotes is the awk program to execute.
print	The print statement is used to output the select fields
$1	Refers to the first field (username username in this case)
$6	Refers to the sixth field (home directory in this case)
$NF	Refers to the last field
/etc/passwd	The input file name/path

BUILT-IN VARIABLES

$0	Reference the whole record/line
$N	Reference Nth field (i.e $1, $2, $3....$NF first, second, third and last field)
FS	Field separator of input file (default whitespace " ")
NR	Line number records
NF	Number of fields in current record
OFS	Output data field separator (default whitespace " ")
ORS	Ouput data record separator (default newline "\n")
RS	Input record separator (default newline "\n")
FILENAME	Name of the input file

ESCAPE SEQUENCE

\	Escape character
\b	Backspace
\f	Form feed
\n	Newline (line feed)
\r	Carriage return
\t	Horizontal tab
\v	Vertical tab

ARITHMATIC OPERATORS

+	Addtion
_	Subtraction
*	Multiplication
/	Division
++	Increment
--	Decrement
%	Modulo

ASSIGNMENT OPERATORS

=	Assignment
+=	Addition and assignment
-=	Subtraction and assignment
*=	Multiplication and assignment
/=	Division and assignment
%=	Modulo and assignment

COMPARISON OPERATORS

==	Equal to
!=	Not equal to
>	Greater than
<	Less than
>=	Greater than or equal to
<=	Less than or equal to

BOOLEAN OPERATOR

&&	Logical AND
||	Logical OR
!	Logical NOT
?:	Ternary Operator

CONDITIONAL OPERATOR

?:	Ternary Operator

REGEX METACHARACTERS

.	Match any Character Except New Line
?	Match 0 or One characters
*	Match 0 or More characters
+	Match 1 or More characters
^	Beginning of line.
$	End of line.
^$	Empty line.
\<	Start of word.
\>	End of word.

ENVIRONMENT VARIABLES

FNR	Represents the record number within the current input file.
CONVFMT	The format used for converting numbers to strings. (default %.6g)
SUBSEP	The separator used for multidimensional array subscripts (default 034)
OFMT	Controls the format used for numeric output (default %.6g)
ARGC	Stores the number of command-line arguments passed to awk
ARGV	Contains the command-line arguments passed to awk as an array
ENVIRON	Provides access to the environment variables as an associative array
RSTART	Stores the starting position of the most recent string match or substitution operation in awk.
RLENGTH	Holds the length of the most recent string match or substitution operation in awk.
ARGIND	Represents the index of the current input file being processed in awk.
IGNORECASE	Ignore case
ERRNO	Stores the error number associated with the most recent system error.
FIELDWIDTHS	Specifies a whitespace-separated list of field widths to split input records into fixed-width fields.

FUNCTIONS

index(s,t)	Position in string s where string t occurs, 0 if not found
length(s)	Length of string s (or $0 if no arg)
rand	Random number between 0 and 1
substr(s,index,len)	Return len-char substring of s that begins at index (counted from 1)
srand	Set seed for rand and return previous seed
int(x)	Truncate x to integer value
split(s,a,fs)	Split string s into array a split by fs, returning length of a
match(s,r)	Position in string s where regex r occurs, or 0 if not found
sub(r,t,s)	Substitute t for first occurrence of regex r in string s (or $0 if s not given)
gsub(r,t,s)	Substitute t for all occurrences of regex r in string s
system(cmd)	Execute cmd and return exit status
tolower(s)	String s to lowercase
toupper(s)	String s to uppercase
getline	Set $0 to next input record from current input file.

FORMAT SPECIFIERS

%c	ASCII character
%d	Decimal integer
%e, %E, %f	Floating-point format
%o	Unsigned octal value
%s	String
%%	Literal % character

AWK FOR-IN LOOP EXAMPLE

```
awk 'BEGIN {
    assoc["key1"] = "val1"
    assoc["key2"] = "val2"
    for (key in assoc)
        print assoc[key];
}'
```

AWK WHILE LOOP EXAMPLE

```
awk 'BEGIN {
    while (a < 10) {
        print "- " " concatenation: " a
        a++;
    }
}'
```

AWK FOR LOOP EXAMPLE

```
awk 'BEGIN {
    for (i = 0; i < 10; i++)
        print "i=" i;
}'
```

AWK DO LOOP EXAMPLE

```
awk '{
    i = 1
    do {
        print $0
        i++
    } while (i <= 5)
}' /etc/passwd
```

AWK IF-ELSE STATEMENT

```
awk -v count=2 'BEGIN {
    if (count == 1)
        print "Yes";
    else
        print "Huh?";
}'
```

AWK IF-ELSE STATEMENT

```
awk -v count=2 'BEGIN {
    print (count==1) ? "Yes" : "Huh?";
}'
```

AWK SWITCH-CASE

```
awk '{
    caseValue = $1;
    switch (caseValue) {
        case "apple":
            print "It's an apple!";
            break;
        case "banana":
            print "It's a banana!";
            break;
        default:
            print "Unknown fruit!";
    }
}' fruits.txt
```

AWK ARRAYS

```
awk 'BEGIN {
    arr[0] = "foo";
    arr[1] = "bar";
    print(arr[0]); # => foo
    delete arr[0];
    print(arr[0]); # => ""
}'
```

AWK MULTI-DIMENSIONAL ARRAY

```
awk 'BEGIN {
    multidim[0,0] = "foo";
    multidim[0,1] = "bar";
    multidim[1,0] = "baz";
    multidim[1,1] = "boo";
}'
```